PROJECT 1

RAG Chatbot — Local Machine Implementation Plan

PDF Q&A System with FastAPI + LangChain + FAISS + Streamlit

Python 3.10+	Gemini AI	LangChain	FAISS	FastAPI	Streamlit

Total Estimated Build Time: 10-14 Days | Difficulty: Intermediate

1. Prerequisites & Local Environment Setup

Before writing a single line of code, ensure your machine has all required tools installed and configured correctly. Missing any of these will cause cryptic errors mid-build.

1.1 System Requirements

Requirement	Minimum	Recommended	Notes
Python	3.10	3.11 or 3.12	Use pyenv to manage versions
RAM	8 GB	16 GB	FAISS in-memory indexing is RAM-heavy
Disk Space	5 GB free	10 GB free	For FAISS indexes + model caches
OS	Windows 10 / macOS 12 / Ubuntu 20.04	macOS 14 / Ubuntu 22.04	All tested
CPU	Any modern x64	4+ cores	For async FastAPI workers
Internet	Required at setup	Stable broadband	For Gemini API calls

1.2 Required Software to Install

- Python 3.10+ — python.org/downloads (or via pyenv: pyenv install 3.11.9)
- Git — git-scm.com (for version control)
- VS Code — code.visualstudio.com (recommended IDE)
- Node.js 18+ — only needed if you want to run the docx skill
- A terminal — PowerShell (Windows) / Terminal (macOS) / bash (Linux)

1.3 Get Your Free Google API Key

Gemini Flash (LLM) and text-embedding-004 (embeddings) are both FREE. You only need one API key for both.

- Go to: aistudio.google.com/app/apikey
- Sign in with Google → Create API Key → Copy it
- Free limits: 15 requests/min, 1M tokens/day for Gemini Flash
- text-embedding-004: 1500 requests/min — more than enough for development

⚠️ **Never hardcode the API key in your Python files. Always load it from .env**

2. Project Folder Structure

Create this exact folder layout before writing any code. A clean structure prevents import errors and makes the codebase navigable.

```
rag-chatbot/
├── app/
│   ├── main.py            # FastAPI app entry point + all API routes
│   ├── rag_pipeline.py    # Document load → chunk → embed → FAISS index
│   ├── chains.py          # RetrievalQA, MapReduce, Refine, Agent chains
│   ├── memory.py          # Per-session memory management
│   └── evaluator.py       # RAGAS evaluation functions
├── frontend/
│   └── streamlit_app.py   # All Streamlit UI screens
├── faiss_index/           # Saved FAISS indexes — gitignore this folder
├── tests/
│   ├── test_pipeline.py   # pytest unit tests for RAG pipeline
│   └── test_api.py        # pytest tests for FastAPI endpoints
├── .env                   # API keys — NEVER commit to Git
```

```
├── .gitignore              # Must include .env and faiss_index/
├── requirements.txt        # All dependencies with pinned versions
├── Dockerfile              # Container config for deployment
└── README.md               # Project documentation (use template from doc)
```

Run this command to create the structure instantly:

```
mkdir -p rag-chatbot/{app,frontend,faiss_index,tests}
cd rag-chatbot
touch app/{main.py,rag_pipeline.py,chains.py,memory.py,evaluator.py}
touch frontend/streamlit_app.py tests/{test_pipeline.py,test_api.py}
touch .env .gitignore requirements.txt README.md
```

3. Dependencies & Installation

3.1 requirements.txt — Complete List

```
# Core LLM & RAG
langchain>=0.2.0
langchain-google-genai
langchain-community
faiss-cpu

# API & Server
fastapi>=0.110
uvicorn
python-multipart       # Required for FastAPI file uploads

# Frontend
streamlit>=1.30

# Document Processing
pypdf2
pdfplumber
unstructured           # For complex/scanned PDFs

# Utilities
python-dotenv
tiktoken               # Token counting

# Evaluation
ragas

# Testing
pytest
httpx                  # For async FastAPI test client
```

3.2 Installation Commands

```
# Step 1: Create virtual environment (always use venv!)
python -m venv venv

# Step 2: Activate it
source venv/bin/activate          # macOS / Linux
venv\Scripts\activate             # Windows PowerShell

# Step 3: Upgrade pip first
pip install --upgrade pip

# Step 4: Install all dependencies
pip install -r requirements.txt

# Step 5: Verify key installs
python -c 'import langchain; print(langchain.__version__)'
```

```
python -c 'import faiss; print("FAISS OK")'
python -c 'import fastapi; print(fastapi.__version__)'
```

3.3 .env File Setup

```
# .env — place in project root, never commit
GOOGLE_API_KEY=your_gemini_api_key_here

# .gitignore — must include:
.env
faiss_index/
__pycache__/
*.pyc
venv/
.DS_Store
```

4. Implementation Plan — 11 Stages

Build the pipeline stage by stage. Each stage is independently testable. Do not skip ahead — each stage's output feeds the next.

Phase 1: Document Loading (Stage 1) ⏱ Day 1 — ~2 hours

- Install and import: PyPDFLoader, WebBaseLoader, DirectoryLoader from langchain_community
- Write a load_document(path) function that auto-detects file type (PDF, TXT, DOCX, URL)
- Always extract and store metadata: filename, page_number, total_pages
- Test by loading a sample PDF and printing documents[0].page_content and documents[0].metadata
- Handle edge cases: empty PDF, password-protected PDF (wrap in try/except)

```
from langchain_community.document_loaders import PyPDFLoader
loader = PyPDFLoader('sample.pdf')
documents = loader.load()
print(f'Loaded {len(documents)} pages')
print(documents[0].metadata)  # {'source': 'sample.pdf', 'page': 0}
```

Phase 2: Text Chunking (Stage 2) ⏱ Day 1 — ~2 hours

- Start with RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200) — this is your default
- Add metadata to every chunk: source filename, page number, chunk index
- Print chunk count after splitting — expect ~3-10x more chunks than pages for typical docs
- Experiment: try chunk_size=500 and chunk_size=2000 on same PDF and compare retrieval quality
- Implement TokenTextSplitter as an alternative (import tiktoken first)

Phase 3: Embedding Generation (Stage 3) ⏱ Day 2 — ~2 hours

- Initialize GoogleGenerativeAIEmbeddings(model='models/text-embedding-004')
- Test with a single sentence: vector = embeddings.embed_query('test') — verify len(vector) == 768
- Cache embeddings to disk — never re-embed the same document twice
- Add logging: log embedding time and dimension size for every run
- Alternative for offline: use HuggingFaceEmbeddings('sentence-transformers/all-MiniLM-L6-v2')

Phase 4: FAISS Vector Database (Stage 4) ⏱ Day 2 — ~2 hours

- Build FAISS index: vectorstore = FAISS.from_documents(chunks, embeddings)
- Save to disk immediately: vectorstore.save_local('faiss_index/') — critical for performance
- Load from disk on subsequent runs: FAISS.load_local(..., allow_dangerous_deserialization=True)
- Test: query = vectorstore.similarity_search('your question', k=4) — verify 4 chunks returned
- Implement add_documents() for incremental updates without rebuilding the entire index

Phase 5: Retrieval Strategies (Stage 5) ⏱ Day 3 — ~3 hours

- Implement all 4 retriever types: similarity, MMR, score_threshold, and self-query
- MMR retriever config: search_type='mmr', search_kwargs={'k': 5, 'fetch_k': 20}
- Test MMR vs similarity on same query — MMR avoids returning duplicate chunks
- Add hybrid search: combine BM25 + FAISS using EnsembleRetriever
- Add contextual compression: post-retrieval, LLM extracts only the relevant sentence

Phase 6: LangChain Chains (Stage 6) ⏱ Day 3-4 — ~4 hours

- Build RetrievalQA chain first: the core Q&A pipeline (return_source_documents=True)
- Add ConversationalRetrievalChain: same as RetrievalQA but with memory attached
- Test with 5 different questions — verify answers cite source page numbers
- Build SequentialChain: chain A (extract topic) → chain B (answer in that context)
- All chains must handle LLM errors gracefully with try/except and fallback messages

Phase 7: Map-Reduce & Refine (Stage 7) ⏱ Day 4 — ~3 hours

- Map-Reduce: chain_type='map_reduce' — use for summarising long documents (100+ pages)
- Refine: chain_type='refine' — use for Q&A where answer spans multiple sections
- Test both on a 50+ page PDF — compare answer quality and response time
- Map-Reduce is faster (parallel), Refine is higher quality (sequential)
- Add to your /summarise endpoint with method='map_reduce' or method='refine' param

Phase 8: Conversation Memory (Stage 8) ⏱ Day 5 — ~2 hours

- Start with ConversationBufferWindowMemory(k=5) — keeps last 5 turns
- Attach to ConversationalRetrievalChain: chain = ConversationalRetrievalChain.from_llm(..., memory=memory)
- Test multi-turn: Q1 asks about revenue, Q2 asks 'what was that again?' — verify context maintained
- Add session-based memory: store separate memory per session_id using a Python dict
- For scale: replace in-memory dict with Redis for persistent cross-restart sessions

Phase 9: RAG Agent with Tools (Stage 9) ⏱ Day 5-6 — ~4 hours

- Wrap your RetrievalQA chain as a LangChain Tool with name='PDF_QA'
- Add web_search_tool using Tavily (free tier) for questions outside the PDF
- Add calculator_tool using LangChain's built-in calculator
- Create ReAct agent: agent = create_react_agent(llm, tools, react_prompt)
- Set max_iterations=5 and handle AgentFinish + AgentError — never let agent run infinitely

Phase 10: FastAPI REST Endpoints (Stage 10) ⏱ Day 6-7 — ~5 hours

- Build all 7 endpoints: POST /upload, POST /ask, POST /ask/stream, POST /summarise, GET /history/{id}, DELETE /clear/{id}, GET /health
- Use FastAPI's UploadFile for multipart PDF uploads in /upload
- Add streaming support in /ask/stream using StreamingResponse with Server-Sent Events
- Add request validation with Pydantic models for all request/response schemas
- Test every endpoint with curl or the auto-generated Swagger UI at localhost:8000/docs

Phase 11: Streamlit Frontend (Stage 11) ⏱ Day 8-9 — ~6 hours

- Screen 1 — Upload: drag-and-drop with st.file_uploader, progress bar, show chunk + page counts
- Screen 2 — Chat: message bubbles using st.chat_message, source citations with page numbers
- Screen 3 — Summarise: radio button for Map-Reduce vs Refine, show word count of summary
- Screen 4 — Settings: sliders for chunk_size, chunk_overlap, k (top-k), temperature
- Screen 5 — Debug: show retrieved chunks, similarity scores, token usage per query
- Sidebar: upload history, model selector (Flash vs Pro), clear session button

5. Evaluation with RAGAS

RAGAS is the standard RAG evaluation framework. This section is what separates a portfolio project from a production system. Implement all 5 metrics.

5.1 RAGAS Metrics & Target Scores

Metric	What It Measures	Target	Fix If Failing
Faithfulness	Answer supported by retrieved context? No hallucinations?	> 0.85	Lower temperature, add 'say I don't know' to prompt
Answer Relevancy	Does the answer address the question asked?	> 0.80	Improve retriever, use MMR, re-rank results
Context Precision	Are retrieved chunks relevant to the question?	> 0.75	Add re-ranking, tighten score threshold
Context Recall	Were ALL needed chunks retrieved?	> 0.70	Increase k or switch to MMR retriever
Answer Correctness	Is the answer factually correct vs ground truth?	> 0.80	Needs a hand-labelled test dataset

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision

# dataset must be a HuggingFace Dataset with columns:
# question, answer, contexts (List[str]), ground_truth
result = evaluate(dataset, metrics=[faithfulness, answer_relevancy, context_precision])
print(result.to_pandas())   # DataFrame with scores per question
```

5.2 Common Problems & Fixes

Problem	Root Cause	Fix
Wrong answer, no hallucination	Retrieval missed relevant chunks	Increase k, use MMR, add re-ranking
Answer contains hallucination	LLM making things up	Lower temperature to 0.1, add 'if not in context, say so' to prompt
Slow response (>5s)	Re-embedding on every request	Save/load FAISS from disk; use async FastAPI
Answer cut off mid-sentence	chunk_overlap too small	Increase chunk_overlap to 300; check max_tokens
Multiple PDFs mixed up	Missing source metadata filter	Add filter={'source': 'file.pdf'} to retriever
Memory growing too large	BufferMemory accumulating all turns	Switch to ConversationSummaryMemory

6. Testing Plan

6.1 Test Scenarios to Cover

Test Type	Scenario	Expected Result
Happy path	Upload valid PDF → ask question	Answer returned with source citation + page number
Multi-turn	Q2 references context from Q1	Memory chain returns contextually correct answer
Out-of-scope	Ask question not in the PDF	Returns: 'This information is not found in the document'
Long document	200+ page PDF	Map-Reduce or Refine chain used; no timeout
Multi-PDF	Question requires info from 2 PDFs	Correct answer synthesised from both sources
Edge — empty PDF	Upload PDF with no text	Graceful error: 'No extractable text found'
Security	Prompt injection attempt	System prompt holds; no instruction leakage
Performance	100 concurrent users (locust)	Avg response time < 5s; no 500 errors

6.2 Running Tests

```
# Install test tools (if not in requirements.txt)
pip install pytest httpx

# Run all tests
pytest tests/ -v

# Run with coverage report
pytest tests/ --cov=app --cov-report=html

# Load testing with locust
pip install locust
locust -f tests/locustfile.py --host=http://localhost:8000
```

7. Running the Project Locally

7.1 Start the FastAPI Backend

```
# From project root, with venv activated:
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# --reload: auto-restart on code changes (dev only)
# Visit: http://localhost:8000/docs for Swagger UI
# Visit: http://localhost:8000/health to verify it's running
```

7.2 Start the Streamlit Frontend

```
# In a SEPARATE terminal (keep FastAPI running):
streamlit run frontend/streamlit_app.py

# Default port: http://localhost:8501
# Custom port:  streamlit run frontend/streamlit_app.py --server.port 8502
```

7.3 Quick Smoke Test

- Open http://localhost:8000/health — expect: {"status": "ok"}
- Open http://localhost:8501 — Streamlit app loads
- Upload a PDF via Screen 1 — verify chunk count displayed
- Ask a question about the PDF — verify answer + source citation
- Ask the same question twice — verify second response is faster (cached index)

7.4 Environment Variables Quick Reference

Variable	Where to Get It	Required?
GOOGLE_API_KEY	aistudio.google.com/app/apikey	Yes — both LLM and embeddings
TAVILY_API_KEY	app.tavily.com (free: 1000 req/month)	Only for Agent web search

8. Recommended Build Timeline

Day	Focus	Deliverable / Milestone
Day 1	Setup + Document Loading + Chunking	Can load any PDF and print chunks with metadata
Day 2	Embeddings + FAISS Index	Can build, save, and load a FAISS index
Day 3	Retrieval + Basic Chain	Can answer questions from a PDF with source citations
Day 4	Map-Reduce + Refine + Memory	Can summarise long docs; chatbot remembers context
Day 5	Agent + Tools	Agent decides when to search PDF vs web vs calculator
Day 6-7	FastAPI Endpoints	All 7 REST endpoints working and tested via /docs
Day 8-9	Streamlit Frontend	Full UI with upload, chat, summarise, debug screens
Day 10	RAGAS Evaluation	All 5 metrics measured; scores > targets

Day	Focus	Deliverable / Milestone
Day 11-12	Testing + Bug Fixes	All test scenarios passing in pytest
Day 13-14	README + Dockerise + Polish	Docker container runs; README complete with demo GIF

9. Quick Reference Card

Key Commands

Action	Command
Activate venv (macOS/Linux)	source venv/bin/activate
Activate venv (Windows)	venv\Scripts\activate
Install dependencies	pip install -r requirements.txt
Run FastAPI	uvicorn app.main:app --reload
Run Streamlit	streamlit run frontend/streamlit_app.py
Run tests	pytest tests/ -v
Check API docs	http://localhost:8000/docs

Key Config Values to Remember

Parameter	Recommended Value	Why
chunk_size	1000	Fits in LLM context, captures full ideas
chunk_overlap	200	20% overlap — prevents answers split across chunks
k (top-k retrieval)	4-5	More context without overloading LLM
temperature	0.1-0.3	Low = more factual, less hallucination for Q&A
max_iterations (agent)	5	Prevents infinite ReAct loops
memory window k	5	Keeps last 5 turns — good balance of context vs tokens

Good luck with the build! Follow the stages in order and test each one before moving on.